



HEALTHCAREADK

END-TO-END AGENTIC DATA ARCHITECTURE

— by  **Farhan Shaikh**

SQL Server · SSIS · Power BI · Python · FastAPI · Claude API · GitHub

✦ **Powered by Claude AI**





WHAT IS HEALTHCAREADK?

A fully integrated data + AI reference implementation on synthetic healthcare data — demonstrating a production-grade agentic architecture from raw data ingestion through AI-powered insights.

DOMAIN

Patients, Providers, Claims, Labs, Prescriptions, Facilities, Payers, Financials

DATA

Fully synthetic — zero real PII. Built as a portfolio / reference impl.

AI LAYER



7 Claude agents run by an orchestrator with schema-level DB isolation

REUSABILITY

`engine/` is domain-agnostic; swap `client/` per domain

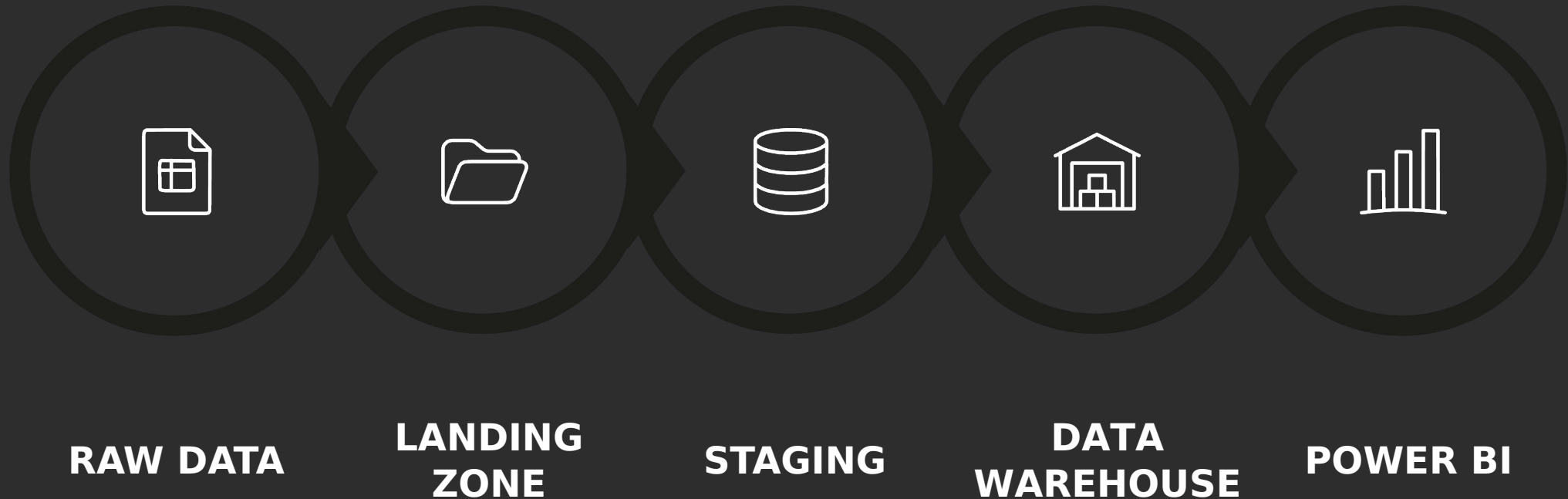
PROJECT PHASES

ALL 8 PHASES COMPLETE

#	Phase	What Was Built
1	Foundation & Synthetic Data	CSV / JSON / Excel generation for all 8 healthcare entities
2	Data Warehouse Design	Star schema — Facts (Claims, Encounters, Rx, Financial) + Dims
3	ETL Pipelines	Raw → staging → DW via stored procedures; SSIS thin wrappers
4	Power BI Reporting	Dashboards: YoY KPIs, denial trends, provider performance
5	Claude AI Layer	MCP servers (sqlserver, shell, file) · RAG · Skills · 7 Hooks
6	Multi-Agent Architecture	7 agents, per-agent SQL logins, token budgets, orchestration
7	Smart Caching	Prompt caching (L1) + SQL response cache (L2) — dw.QueryCache
8	Semantic Query Cache	Local embeddings + cosine similarity + Haiku equivalence gate (L3)

DATA PIPELINE

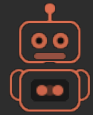
Every row flows through the staging layer before reaching the data warehouse — no raw data bypasses stg.*.



The ETL stored procedure `usp_ETL_RunAll` orchestrates every transformation step. The Claude AI layer sits above the warehouse, querying through scoped views — never touching raw or staging tables directly.

MULTI-AGENT ARCHITECTURE

OrchestratorAgent routes every request, enforces token budgets, and manages cache checks. Each domain agent operates under its own SQL Server login — schema isolation is enforced at the database engine level.



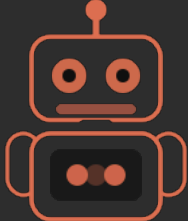
ClaimsAgent
(claims schema)
per-agent SQL login
and schema-level
GRANT/DENY



ClinicalAgent
(patients/labs/rx)
per-agent SQL login
and schema-level
GRANT/DENY

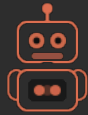


ReportingAgent
(DW views read-only)
per-agent SQL login
and schema-level
GRANT/DENY



OrchestratorAgent

- Routes
- Budgets
- Caches
- L2+L3 cache check
- multi-hop result merge



FinancialAgent
(financials schema)
per-agent SQL login
and schema-level
GRANT/DENY



ProviderAgent
(providers/facilities)
per-agent SQL login
and schema-level
GRANT/DENY



ETLAgent
(staging schema)
per-agent SQL login
and schema-level
GRANT/DENY

PER-AGENT SQL LOGIN

Each agent authenticates as `agent_claims`, `agent_clinical`, etc.
— GRANT/DENY enforced in `sql/10_agent_permissions.sql`

TOKEN BUDGET ENFORCEMENT

OrchestratorAgent checks budget before dispatch AND mid-loop.
Exhaustion escalates gracefully — never runs unbounded.

MULTI-HOP RESULT MERGE

Cross-agent queries are decomposed, routed to the appropriate agents, and results merged by the orchestrator before responding.

THREE-LAYER CACHING ARCHITECTURE

Zero-token hits on exact and semantically equivalent repeated queries — every layer has a distinct mechanism and failure mode.

1

L1 — PROMPT CACHE

cache_control: ephemeral on system prompts · Anthropic-side · No DB required · Reduces input token cost within the 5-min cache window

2

L2 — EXACT RESPONSE

dw.QueryCache keyed by SHA-256(agent + query) · Per-agent TTL 15-60 min · Zero API tokens on identical repeated queries

3

L3 — SEMANTIC CACHE

Local embeddings (all-MiniLM-L6-v2) + cosine ≥ 0.80 + Haiku equivalence gate · Catches paraphrases — *"payments in ohio"* vs *"payments in OH"*

❏ L3 cosine similarity score produces a candidate only — Haiku verification is mandatory and **fails closed**. The system never serves a cached response on score alone.

SECURITY & GUARDRAILS

Principle of least privilege enforced at every layer — from database engine to API surface to Claude hook scripts.

SQL SCHEMA ISOLATION

Per-agent SQL Server logins · GRANT/DENY per schema ·
Enforced by the DB engine, not application logic

7 CLAUDE HOOKS

`guard_file_read.py` blocks `.env` reads · `guard_query.py` blocks
DML/DDI on `dw.*` · writes scoped to `powerbi/tmdl/` or `sql/`

TOKEN BUDGETS

Per-agent per-session cap in YAML config · Checked before
dispatch AND mid-loop · Exhaustion escalates gracefully

API AUTHENTICATION

POST /chat requires `X-API-Key` · Session registry rejects
orphaned/replayed sessions · CORS restricted to configured origins

SECRET MANAGEMENT

`.env` gitignored + guarded by hook · Agent passwords via env
vars only · `sqlcmd` passwords via `SQLCMDPASSWORD`, never `-P` flag

HIPAA CONTROL MAP

`docs/hipaa_control_map.md` maps 30 features to § 164.308 /
310 / 312 — Technical, Administrative, and Physical safeguards

ENGINE / CLIENT SPLIT



Zero domain knowledge lives in engine/ — copy it wholesale to any new project.
Swap out client/ entirely to target a new business domain.

ENGINE/ — DOMAIN AGNOSTIC

BASE.PY

Agentic loop — tool execution, iteration cap, budget recording

BUDGET_TRACKER.PY

Token usage logger → `dw.AgentUsageLog` per session

CACHE.PY

L2 exact cache + L3 semantic cache (embed, similarity, gate)

EMBEDDINGS.PY

Lazy sentence-transformers singleton — all-MiniLM-L6-v2

CLIENT/ — HEALTHCARE DOMAIN PACK

AGENTS/

7 agents (YAML + Python), orchestrator, tools

API/ + FRONTEND/

FastAPI chat endpoint + Vanilla HTML/JS/CSS UI (no build step)

MCP/ + SQL/

3 MCP servers · DDL, ETL procs, views, permissions (00-13)

SCRIPTS/ + DOCS/

Data gen, login deploy, 7 hook scripts · Phase designs, HIPAA map, onboarding guide

📄 Replace `client/` entirely for a new domain. See `docs/new_client_onboarding.md` for the step-by-step guide.

CLAUDE TOOLS

The Claude Code tooling that powers this project — each layer gives the agents a different capability, scoped and guarded.

CLAUDE.md

Project "constitution" — domain, phases, agent roles, guardrails, directory map.
Read first by Claude on every session.

MCP SERVERS

sqlserver · shell · file (FastMCP)
SQL exec, dtexec/pbi-tools, file I/O —
the agents' hands on the system.

SKILLS

Reusable task specs: claims-summary,
financial-yoy, abnormal-labs.
Packaged know-how agents invoke.

TOOLS

SQL tools (pyodbc), file I/O tools,
shell tools (dtexec, python, pbi-tools,
sqlcmd). Defined per agent with an
allowlist — an agent only sees the
tools its config grants it.

HOOKS

7 hooks in .claude/settings.json.
PreToolUse guards block .env reads &
DML/DDI on dw.*; PostToolUse hooks
auto-run ETL & surface deploy output.

TECH STACK



End-to-end from raw synthetic files to AI-powered conversational insights — every layer is explicit, version-pinned, and replaceable.

Layer	Technology	Notes
Raw Data	Synthetic CSV / JSON / Excel	scripts/generate_all.py — 8 entities
Database	SQL Server (Express / Developer edition)	Mixed Mode Auth · ODBC Driver 17
ETL	Python stored procedures + SSIS thin wrappers	usp_ETL_RunAll · staging layer always enforced
BI	Power BI Desktop + TMDL text model	pbi-tools CLI for model deployment
AI Model	Claude — Sonnet 4.6 (agents) · Haiku (cache gate)	claude-sonnet-4-6 · claude-haiku-4-5-20251001
Agent Framework	Claude Agent SDK + MCP via FastMCP	3 live MCP servers (sqlserver, shell, file)
Caching	Prompt cache + SQL response cache + local embeddings	sentence-transformers all-MiniLM-L6-v2
Chat API	FastAPI + uvicorn	POST /chat · X-API-Key · CORS · session registry
Frontend	Vanilla HTML / JS — marked.js + DOMPurify	No build step · file:// or localhost:8000
Dev Environment	VS Code + Claude Code CLI	MCP wired via .mcp.json · 7 PreToolUse/PostToolUse hooks

ONBOARDING A NEW DOMAIN

Copy engine/ untouched · swap client/ · point the framework at a new warehouse.

1 COPY

Keep engine/ + tests/, gut client/.

2 DATABASE

New domain DW. Reuse usage + cache
DDL unchanged.

3 AGENTS

One per domain + orchestrator: YAML
config + a ~10-line .py.

4 WIRE

Update AGENT_MODULE_MAP. MCP paths
self-heal.

5 SECURE

Per-agent SQL logins, schema GRANT /
DENY, .env secrets.

6 RUN

Unit + permission tests green, then
uvicorn client.api.main:app.

DOMAIN-AGNOSTIC BY DESIGN

same engine/, zero edits

Healthcare Claims · Clinical · Financial · Provider

Retail Sales · Inventory · Customer · Finance